INTERNSHIP PROJECT REPORT

WEB APPLICATION SECURITY TESTING

Internship Organization: ApexPlanet Software Pvt. Ltd.

Internship Domain: Cyber Security

Project Title: Web Application Security Testing using DVWA

Submitted By: K. Saathvika

Tools Used: Kali Linux, Damn Vulnerable Web Application (DVWA), Burp Suite, Chromium

Internship Task: Web Application Security Testing

Year: 2026

1. INTERNSHIP OVERVIEW

The cybersecurity internship offered by ApexPlanet Software Pvt. Ltd. provided an opportunity to gain practical knowledge of web application security and vulnerability assessment.

As part of this internship, I worked on identifying and understanding common web application vulnerabilities using the Damn Vulnerable Web Application (DVWA). The practical activities were performed in a controlled lab environment using Kali Linux and Burp Suite.

The internship focused on understanding how security vulnerabilities occur, how they can be tested, and what security measures can be implemented to reduce potential risks.

Internship Organization: ApexPlanet Software Pvt. Ltd.

Domain: Cyber Security

Project: Web Application Security Testing

Testing Environment: Kali Linux with DVWA

Tools: Burp Suite and Chromium

Internship Learning Outcomes

During the internship, I gained practical exposure to:

- Understanding common web application vulnerabilities.
- Identifying security weaknesses in a controlled testing environment.
- Using Burp Suite to intercept and analyze HTTP requests.
- Testing SQL Injection, Cross-Site Scripting, CSRF, and File Inclusion vulnerabilities.
- Understanding the importance of HTTP security headers.
- Learning basic techniques to improve web application security.

2. PROJECT INTRODUCTION

Web application security is an important aspect of cybersecurity that focuses on protecting websites and web-based applications from unauthorized access, data theft, and malicious attacks.

This project focuses on testing web application vulnerabilities using the Damn Vulnerable Web Application (DVWA), an intentionally vulnerable web application designed for security testing and learning purposes.

The testing was performed in a controlled local environment using Kali Linux. Burp Suite was used to intercept and analyze HTTP requests and responses, while Chromium was used to access the DVWA application.

The project involved testing different security vulnerabilities, observing their behaviour at different security levels, and understanding how appropriate security mechanisms can help protect web applications.

The practical work helped develop an understanding of vulnerability identification, request analysis, and basic web application security testing techniques.

Scope of the Project

The project covers practical testing of selected web application vulnerabilities in DVWA, including:

1. Cross-Site Scripting (XSS)
2. Cross-Site Request Forgery (CSRF)
3. Local File Inclusion (LFI)
4. Remote File Inclusion (RFI)
5. SQL Injection at different security levels
6. HTTP Security Headers analysis
7. HTTP request testing using Burp Suite Intruder

All testing activities were performed within a local, controlled lab environment for educational purposes.

3. PROJECT OBJECTIVES

The main objective of this project is to understand and perform basic web application security testing using DVWA in a controlled environment.

The specific objectives are:

1. To understand the fundamentals of web application security.
2. To set up and configure the DVWA testing environment in Kali Linux.
3. To identify and test common web application vulnerabilities.
4. To understand Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF).
5. To perform Local File Inclusion (LFI) and Remote File Inclusion (RFI) testing.
6. To analyze SQL Injection vulnerabilities at different security levels.
7. To use Burp Suite for intercepting and analyzing HTTP requests.
8. To perform parameter testing using Burp Suite Intruder.
9. To analyze HTTP response headers and understand their security importance.
10. To understand basic preventive measures for improving web application security.

Overall Goal:

To gain practical knowledge of web application vulnerability assessment and develop an understanding of basic security testing techniques.

Importance of the Project

Web application security testing helps identify weaknesses before attackers can exploit them. It plays an important role in protecting sensitive information, maintaining user privacy, and ensuring the reliability of web applications.

Through this project, practical knowledge was gained about how vulnerabilities affect web applications and how security configurations can reduce risks.

Expected Outcomes

- Improved understanding of common web security vulnerabilities.
- Practical experience with DVWA and Kali Linux.
- Ability to inspect HTTP requests and responses using Burp Suite.

- Knowledge of basic vulnerability testing methods.
- Awareness of security headers and their role in protecting web applications.
- Understanding of the importance of secure coding and proper input validation.

4. TOOLS AND TECHNOLOGIES USED

The following tools and technologies were used to perform web application security testing during the internship.

4.1 Kali Linux

Kali Linux is a Linux-based operating system designed for cybersecurity, penetration testing, and digital forensics. It provided the environment for running DVWA and performing security testing activities.

4.2 DVWA (Damn Vulnerable Web Application)

DVWA is an intentionally vulnerable web application developed for security training and educational purposes. It provides different vulnerability modules that help learners understand common web application security issues.

4.3 Burp Suite

Burp Suite is a web application security testing platform used to intercept, inspect, and modify HTTP requests and responses. It was used for SQL Injection testing, request analysis, and Intruder testing.

4.4 Chromium Browser

Chromium was used to access the locally hosted DVWA application and interact with its different vulnerability modules.

4.5 Apache Server

Apache HTTP Server was used to host the DVWA application locally and serve its web pages.

4.6 MariaDB Database

MariaDB was used as the database management system for storing and retrieving the data required by DVWA.

4.7 PHP

PHP is a server-side scripting language used by DVWA to process application requests and interact with the database.

4.8 Web Technologies

HTTP requests, HTTP responses, cookies, session handling, and security headers were studied during the testing process.

5. ENVIRONMENT SETUP

Before performing web application security testing, a local testing environment was prepared using Kali Linux and DVWA.

5.1 Kali Linux Setup

Kali Linux was used as the primary operating system for performing cybersecurity practical activities. It provided the required environment and tools for web application security testing.

5.2 Starting Apache and MariaDB Services

Apache and MariaDB services were started to make the DVWA application accessible locally.

The following commands were used:

sudo systemctl start apache2

sudo systemctl start mariadb

5.3 Accessing DVWA

After starting the required services, DVWA was accessed through the Chromium browser using the following local address:

http://localhost/DVWA

5.4 DVWA Login

The DVWA login page was opened, and the default lab credentials were used:

- Username: admin
- Password: password

5.5 Security Level Configuration

DVWA provides different security levels such as Low, Medium, and High. These levels were selected to understand how security controls affect vulnerability testing.

5.6 Burp Suite Configuration

Burp Suite was used as an intercepting proxy to capture and analyze HTTP traffic between the browser and DVWA. The proxy listener was configured at:

127.0.0.1:8080

The environment was successfully prepared for conducting the planned security testing activities.

6. CROSS-SITE SCRIPTING (XSS) TESTING

6.1 Introduction

Cross-Site Scripting (XSS) is a web application vulnerability that allows attackers to inject malicious scripts into web pages viewed by other users. These scripts can execute in the victim's browser and may be used to steal information or perform unauthorized actions.

6.2 Objective

To understand and test Stored XSS and Reflected XSS vulnerabilities using DVWA.

6.3 Tools Used

- Kali Linux
- DVWA
- Chromium Browser

6.4 Procedure

A. Stored XSS Testing

1. Open the DVWA application.
2. Select the Stored XSS module.
3. Enter a test payload in the input field.
4. Submit the input.
5. Observe the browser response.

Test payload:

<script>alert('Stored XSS Attack')</script>

Result: A JavaScript alert box appeared, demonstrating successful Stored XSS execution.

B. Reflected XSS Testing

1. Open the Reflected XSS module in DVWA.
2. Enter the test payload in the input field.
3. Submit the input.
4. Observe the browser response.

Test payload:

<script>alert('Reflected XSS')</script>

Result: A JavaScript alert box appeared, demonstrating successful Reflected XSS execution.

6.5 Conclusion

Both Stored XSS and Reflected XSS were successfully demonstrated in the controlled DVWA environment. This experiment showed the importance of input validation, output encoding, and secure handling of user-supplied data.

7. CROSS-SITE REQUEST FORGERY (CSRF) TESTING

7.1 Introduction

Cross-Site Request Forgery (CSRF) is a web security vulnerability that tricks an authenticated user into performing unwanted actions on a web application without their knowledge or consent.

7.2 Objective

To understand how CSRF attacks work and observe how DVWA handles password-change requests and security tokens.

7.3 Tools Used

- Kali Linux
- DVWA
- Burp Suite
- Chromium Browser

7.4 Procedure

1. Open the DVWA application.
2. Navigate to the CSRF vulnerability module.
3. Observe the password-change form.
4. Enter the current password and a new test password.
5. Submit the form.
6. Observe the response displayed by DVWA.
7. Open Burp Suite and inspect the captured HTTP request.
8. Examine the request parameters and check for the presence of a user token.

7.5 Observation

The application displayed the message "Password Changed" after submitting the password-change form.

The captured HTTP request contained password-related parameters and a user_token value. This demonstrated the presence of a token-based CSRF protection mechanism in the tested request.

7.6 Result

The password-change operation was completed successfully. The request was inspected using Burp Suite, and a CSRF token was observed.

7.7 Conclusion

This experiment helped understand CSRF vulnerabilities and the importance of using unpredictable anti-CSRF tokens to protect sensitive user actions.

8. LOCAL FILE INCLUSION (LFI) TESTING

8.1 Introduction

Local File Inclusion (LFI) is a web application vulnerability that occurs when an application includes files from its local server without properly validating user input. An attacker may exploit this weakness to access sensitive files stored on the server.

8.2 Objective

To identify and demonstrate a Local File Inclusion vulnerability using the DVWA File Inclusion module.

8.3 Tools Used

- Kali Linux
- DVWA
- Chromium Browser

8.4 Procedure

1. Open DVWA in the Chromium browser.
2. Navigate to the File Inclusion module.
3. Set the DVWA security level to Low.
4. Observe the page parameter in the URL.
5. Modify the parameter to test access to a local system file.
6. Submit the modified URL.
7. Observe the server response.

Test URL:

http://localhost/DVWA/vulnerabilities/fi/?page=../../../../../../etc/passwd

8.5 Observation

The application displayed Linux account information from the /etc/passwd file. This indicated that the application allowed access to a local file through an improperly validated page parameter.

8.6 Result

Local File Inclusion was successfully demonstrated in the DVWA testing environment.

8.7 Conclusion

This experiment demonstrated the risks of improper file path validation. Applications should use strict input validation, allowlisted file names, and safe file-handling practices to prevent LFI vulnerabilities.

9. REMOTE FILE INCLUSION (RFI) TESTING

9.1 Introduction

Remote File Inclusion (RFI) is a web application vulnerability that occurs when an application allows files from an external server to be included and executed. If exploited, it may allow an attacker to execute malicious code on the target server.

9.2 Objective

To understand and demonstrate Remote File Inclusion using the DVWA File Inclusion module in a controlled environment.

9.3 Tools Used

- Kali Linux
- DVWA
- Chromium Browser
- Python HTTP Server

9.4 Procedure

1. Open DVWA in the Chromium browser.
2. Navigate to the File Inclusion module.
3. Set the DVWA security level to Low

10. SQL INJECTION TESTING – LOW SECURITY

10.1 Introduction

SQL Injection is a web application vulnerability that occurs when user input is directly included in SQL queries without proper validation or parameterization. It may allow attackers to manipulate database queries and access unauthorized information.

10.2 Objective

To identify and demonstrate SQL Injection vulnerability in DVWA at the Low security level using Burp Suite.

10.3 Tools Used

- Kali Linux
- DVWA
- Burp Suite
- Chromium Browser
- MariaDB

10.4 Procedure

1. Open DVWA in Chromium.
2. Select the SQL Injection module.
3. Set the security level to Low.
4. Enter a normal user ID and submit the request.
5. Capture the HTTP request using Burp Suite.
6. Send the request to Burp Repeater.
7. Modify the id parameter with a test SQL Injection payload.
8. Send the modified request and observe the response.

Test Payload:

1' OR '1'='1' #

10.5 Observation

The application returned multiple database records instead of displaying only one user's information. The response included records such as admin, Gordon Brown, Hack Me, Pablo Picasso, and Bob Smith.

10.6 Result

SQL Injection was successfully demonstrated at the Low security level. The application was vulnerable because user input was not safely handled in the SQL query.

10.7 Conclusion

This experiment demonstrated how improperly handled user input can manipulate database queries. Prepared statements, parameterized queries, and input validation are important measures for preventing SQL Injection.

11. SQL INJECTION TESTING – MEDIUM SECURITY

11.1 Introduction

DVWA Medium security introduces additional input-handling mechanisms to make SQL Injection more difficult. It demonstrates how security controls can reduce the risk of malicious SQL query manipulation.

11.2 Objective

To examine how the Medium security level handles SQL Injection attempts and compare its behaviour with the Low security level.

11.3 Tools Used

- Kali Linux
- DVWA
- Burp Suite
- Chromium Browser
- MariaDB

11.4 Procedure

1. Open the DVWA application.
2. Navigate to the SQL Injection module.
3. Change the security level to Medium.
4. Submit a normal user ID.
5. Capture the HTTP request using Burp Suite.
6. Observe the POST request and its parameters.
7. Send the request to Burp Repeater.
8. Modify the ID parameter using the previously tested SQL Injection payload.
9. Send the request and examine the response.

11.5 Observation

The modified request returned an SQL syntax error instead of displaying multiple database records. The error indicated that the application had applied additional input-handling measures, including escaping quote characters.

11.6 Result

The previously successful Low-level SQL Injection payload did not work in the same way at the Medium security level. The application demonstrated additional protection against the tested input.

11.7 Conclusion

This experiment showed that input-handling mechanisms can make SQL Injection more difficult. However, escaping alone should not be considered sufficient protection. Parameterized queries and prepared statements are recommended for secure database operations.

12. SQL INJECTION TESTING – HIGH SECURITY

12.1 Introduction

DVWA High security provides stronger restrictions on user input and changes the way the application accepts user IDs. It demonstrates how additional security controls can reduce the possibility of SQL Injection attacks.

12.2 Objective

To observe the behaviour of the SQL Injection module at the High security level and understand its additional security controls.

12.3 Tools Used

- Kali Linux
- DVWA
- Burp Suite
- Chromium Browser
- MariaDB

12.4 Procedure

1. Open the DVWA application.
2. Navigate to the SQL Injection module.
3. Change the security level to High.

4. Observe the updated interface.
5. Click the link provided to change the user ID.
6. Select a user ID through the separate input page.
7. Submit the selection and observe the application response.
8. Inspect the corresponding request using Burp Suite.

12.5 Observation

At the High security level, the normal ID input field was replaced with a separate link that opened a session-based input page. The application provided a more restricted method for selecting the user ID.

12.6 Result

The High security interface and request-handling behaviour were successfully examined. The application used a different input mechanism compared with the Low and Medium security levels.

12.7 Conclusion

This experiment helped understand how additional input restrictions and session-based handling can improve application security. Secure applications should use parameterized SQL queries and strict server-side validation rather than relying only on interface restrictions.

13. BURP SUITE INTRUDER TESTING

13.1 Introduction

Burp Suite Intruder is a tool used in web application security testing to automate the sending of HTTP requests with different input values. It helps testers examine how an application responds to multiple parameter values.

13.2 Objective

To understand the functionality of Burp Suite Intruder by testing different user ID values in the DVWA SQL Injection module.

13.3 Tools Used

- Kali Linux
- DVWA
- Burp Suite
- Chromium Browser

13.4 Procedure

1. Open DVWA in the Chromium browser.
2. Navigate to the SQL Injection module.
3. Set the security level to Low.
4. Submit a normal user ID.
5. Capture the HTTP request using Burp Suite Proxy.
6. Send the captured request to Intruder.
7. Select the ID parameter as the payload position.
8. Choose the Simple List payload type.
9. Add the test values 1, 2, 3, 4, and 5.
10. Start the Intruder attack.
11. Observe and compare the responses for different values.

13.5 Observation

Burp Suite Intruder automatically sent multiple requests using the configured ID values. The responses were displayed in the attack results window, allowing comparison of the application's behaviour for different inputs.

13.6 Result

The Intruder attack was successfully performed using a list of five test ID values.

13.7 Conclusion

This experiment demonstrated how Burp Suite Intruder can automate parameter testing and help identify unusual application responses during security assessments.

14. HTTP SECURITY HEADERS ANALYSIS

14.1 Introduction

HTTP Security Headers are response headers used by web servers to improve the security of web applications. They help protect against common browser-based attacks such as clickjacking, MIME-type sniffing, and certain content injection attacks.

14.2 Objective

To inspect HTTP response headers and configure basic security headers in the Apache web server.

14.3 Tools Used

- Kali Linux
- Apache HTTP Server
- Chromium Browser
- Developer Tools

14.4 Procedure

1. Open the DVWA application in Chromium.
2. Open Developer Tools and select the Network tab.
3. Reload the DVWA page.
4. Select the localhost request.
5. Open the Headers section and inspect Response Headers.
6. Record the security headers available before configuration.
7. Enable the Apache Headers module.
8. Create an Apache security configuration file.
9. Add the following security headers:

Header always set X-Frame-Options "SAMEORIGIN"

Header always set X-Content-Type-Options "nosniff"

Header always set Content-Security-Policy "default-src 'self'"

10. Enable the configuration and restart Apache.
11. Reload the DVWA page and inspect the response headers again.

14.5 Observation

Before configuration, the following security headers were absent:

- X-Frame-Options
- X-Content-Type-Options
- Content-Security-Policy

After configuration, all three headers appeared successfully in the HTTP response.

14.6 Result

The Apache server was successfully configured to return the three selected security headers.

14.7 Conclusion

This experiment demonstrated the importance of HTTP Security Headers in improving web application security. Proper configuration helps reduce certain browser-based security risks.

HSTS was not configured because the local DVWA environment used HTTP rather than HTTPS.

15. RESULTS AND OBSERVATIONS

The web application security testing project was successfully performed using DVWA in a controlled local environment. Various vulnerability testing techniques were carried out to understand common security weaknesses and observe application behaviour.

15.1 Summary of Experiments

S.No.	Experiment	Result
1	Stored XSS	Successful
2	Reflected XSS	Successful

3	CSRF Request Analysis	Token observed
4	Local File Inclusion	Successful
5	Remote File Inclusion	Successful
6	SQL Injection – Low	Successful
7	SQL Injection – Medium	Payload blocked by input handling
8	SQL Injection – High	Security controls observed
9	Burp Suite Intruder	Successfully performed
10	HTTP Security Headers	Successfully configured

15.2 Key Observations

1. DVWA provided a practical environment for understanding common web application vulnerabilities.
2. XSS testing demonstrated the risks of improper input handling and output encoding.
3. CSRF request analysis showed the presence of a user token in the captured request.
4. File Inclusion testing demonstrated the risks of unsafe file path handling.
5. SQL Injection testing showed differences in application behaviour across security levels.
6. Burp Suite helped intercept, modify, and analyze HTTP requests.
7. Apache security headers were successfully configured and verified.

15.3 Overall Result

The project objectives were achieved through practical testing and analysis. The experiments provided hands-on experience in identifying vulnerabilities, examining HTTP traffic, and applying basic web security configurations.

16. CHALLENGES FACED AND SOLUTIONS

During the web application security testing project, several technical challenges were encountered while setting up the environment and performing the experiments. These challenges provided opportunities to improve troubleshooting and practical cybersecurity skills.

16.1 Environment Configuration

Challenge: Setting up DVWA and ensuring that the required services were running correctly.

Solution: Apache and MariaDB services were started and the local DVWA application was accessed through the Chromium browser.

16.2 Burp Suite Configuration

Challenge: Capturing and analyzing HTTP requests using Burp Suite required proper proxy configuration.

Solution: The proxy listener was configured at 127.0.0.1:8080, and browser requests were inspected through Burp Suite.

16.3 SQL Injection Testing

Challenge: The same SQL Injection payload did not produce the same results at different DVWA security levels.

Solution: The requests and responses were compared using Burp Suite to understand the effect of different input-handling mechanisms.

16.4 Remote File Inclusion

Challenge: Remote file inclusion required additional PHP configuration in the local testing environment.

Solution: The required setting was temporarily enabled for the controlled test and restored to Off after the experiment.

16.5 Security Headers Configuration

Challenge: The required security headers were initially missing from the server responses.

Solution: The Apache Headers module was enabled, and a security configuration file was created. Apache was restarted, and the headers were verified using Chromium Developer Tools.

16.6 Learning Outcomes

These challenges improved practical knowledge of Linux commands, server configuration, HTTP request analysis, vulnerability testing, and troubleshooting techniques.

17. CONCLUSION

The Web Application Security Testing internship project provided valuable practical knowledge of cybersecurity and web application vulnerability assessment.

During this project, various security testing activities were performed using the Damn Vulnerable Web Application (DVWA) in a controlled local environment. The experiments included Stored XSS, Reflected XSS, CSRF request analysis, Local File Inclusion, Remote File Inclusion, SQL Injection at different security levels, Burp Suite Intruder testing, and HTTP Security Headers configuration.

The use of Kali Linux helped in preparing the testing environment, while Burp Suite provided practical experience in intercepting and analyzing HTTP requests and responses. Chromium Developer Tools were also used to inspect HTTP response headers.

The project helped develop an understanding of how web application vulnerabilities occur, how they can be identified, and what security measures can be applied to reduce risks.

It also improved practical skills in Linux commands, web technologies, server configuration, HTTP communication, and security testing tools.

Overall, this internship project was a valuable learning experience that strengthened my interest and understanding of the cybersecurity domain.

Future Scope

Further work can include learning advanced penetration testing techniques, exploring the OWASP Top 10 vulnerabilities, performing secure coding practices, and studying automated vulnerability assessment tools.

18. REFERENCES

The following resources were referred to during the internship project for understanding web application security concepts, tools, and testing procedures.

1. **ApexPlanet Software Pvt. Ltd.** – Internship tasks and learning materials.
2. **Damn Vulnerable Web Application (DVWA)** – Official GitHub Repository. https://github.com/digininja/DVWA
3. **Burp Suite Documentation** – PortSwigger. https://portswigger.net/burp/documentation
4. **Kali Linux Documentation** – Official Documentation. https://www.kali.org/docs/
5. **OWASP Foundation** – Web Application Security Guidelines. https://owasp.org/
6. **OWASP Top 10** – Web Application Security Risks. https://owasp.org/www-project-top-ten/
7. **Apache HTTP Server Documentation**. https://httpd.apache.org/docs/
8. **PHP Documentation**. https://www.php.net/docs.php
9. **MDN Web Docs** – HTTP Security Headers. https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers

Final Note

All practical experiments documented in this report were performed in a controlled local testing environment for educational purposes.